

FOR SOFTWARE ENGINEERS: THE HARNESS, NOT THE CHAT WINDOW

Claude Code

Everything a software engineer needs to know

Forty files edited, then escape twice and all of it gone. Not reverted with git. Rewound, to a state never committed.

That safety net is the single reason it is reasonable to let something else edit your code at all. It also has a gap in it, and the gap is narrower than most people assume.

This sheet is the parts that change how you work: the undo and what it misses, how sessions keep a large codebase out of your context, the three files that make it a team tool, and the failure that costs the most and gets talked about least.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- Why it is a harness rather than a chat window, and why the loop is the new thing
- What checkpointing tracks, and the four things it does not
- How subagents keep the evidence out of your context and return the conclusion
- The three files you commit, and the one you must not
- The stopping failure, and the three things that help

Out of scope

- Hooks, output styles and the status line
- Agent teams, plugins and the plugin marketplace
- Routines, channels and the enterprise controls
- Anything about how this channel is produced, which is not the subject

Each of those deserves its own piece rather than ninety seconds here. Read the checkpointing table first: it is the one that changes what you are willing to let an agent do.

01 What the undo actually covers

Its own file edit tools	Tracked. Recover with <code>/rewind</code>
A bash command	NOT tracked. Recover with <code>git</code>
A background subagent	Usually not tracked. Recover with <code>git</code>
Symlinked or hard linked paths	Skipped on restore. Recover by hand
Age	Snapshots swept after about thirty days
What it restores	Code, conversation, or both, with no commit required

This is session level recovery. The documentation is blunt about what follows and so is this sheet: it is not a replacement for version control.

03 It is a harness, not a chatbot

Think of it as a chat window that knows code and you paste code in and answers back out, and you get a slightly faster autocomplete.

Think of it as a harness and you give it access, a task, and a way to check its own work, and it closes the loop without you.

Code generation is old and was never the bottleneck. THE LOOP IS THE NEW THING: run, read the failure, change approach, run again.

It operates as an agentic harness with access to tools, rather than as a conversational interface that returns text.

Claude Code docs, Overview, checked September 2026

The capability that changed is not producing code but iterating against a check without a human between the attempts.

First principles, not vendor documentation

04 The safety net, and exactly where it leaks

Before each prompt that starts a turn, the state of your code is captured. Run /rewind, or press escape twice on an empty input, and you can restore the code, the conversation, or both. No commit required, and it is saved with the conversation, so you can resume tomorrow and rewind yesterday.

Now the gap. Checkpointing only tracks changes made through its OWN file editing tools. A bash command that rewrites a file is invisible to it. So is a background subagent, usually. Symlinked and hard linked paths are skipped on restore.

Snapshots are swept after about thirty days. This is session level recovery and it is not version control, which is the sentence to remember before you rely on it.

Checkpoints are created before each prompt and can restore code, conversation, or both, without requiring a commit.

Claude Code docs, Checkpointing, checked September 2026

Checkpointing tracks only changes made through its own file editing tools, and does not capture changes made by bash commands.

Claude Code docs, Checkpointing, checked September 2026

Symbolic and hard linked paths are skipped during restore, and snapshots are cleaned up after roughly thirty days.

Claude Code docs, Checkpointing, checked September 2026

05 How sessions communicate, which is why a big codebase does not drown you

The expensive part of searching a large codebase is not the searching. It is the thousands of lines of file content that land in your context afterwards and stay there for the rest of the session.

So do not do it in your own window. Ask for a subagent. It gets its own isolated context, reads the forty files there, and returns a conclusion.

What comes back is the ANSWER, not the pile of evidence. That is the whole mechanism, and it is the part nobody explains well.

A subagent runs with its own separate context window and returns its result to the calling session.

Claude Code docs, Subagents, checked September 2026

Delegating a broad search keeps the intermediate file contents out of the calling session, which is the cost being avoided.

First principles, not vendor documentation

06 The three files you commit, and the one you must not

CLAUDE.md, committed, gives everyone the same project context on their first run.

.claude/settings.json, committed, carries the commands the team has agreed are safe, already approved.

.claude/skills/, committed, brings your review checklist along with the clone.

.claude/settings.local.json is PERSONAL. It goes in .gitignore. You are sharing the standards, not the conversation.

CLAUDE.md is read as project memory and is intended to be checked into the repository.

Claude Code docs, Memory and CLAUDE.md, checked September 2026

Settings are split into a shared project file and a local file, and the local file is intended to stay out of version control.

Claude Code docs, Permission modes, checked September 2026

A skill is a directory with a SKILL.md in it, so committing the directory distributes the skill.

Claude Code docs, Skills, checked September 2026

07 A skill is just a file

Create a SKILL.md with instructions in it and it is in the toolkit. No framework, no registration, nothing to compile.

Invoke it by name. A short one that attacks a plan, finds the assumption it rests on, and asks which failure the design does not handle is more useful than it sounds.

The useful ones are boring and specific: your code review checklist, your release steps, how this team writes tests.

A skill is defined by a SKILL.md file with name and description frontmatter, and requires no build step or registration.

Claude Code docs, Skills, checked September 2026

08 The failure that costs the most

You give it a standing instruction. Work through this list, do not stop, and if something is blocked skip it and carry on.

It does three items. The fourth needs a human for one click. And it stops. Not that item, THE WHOLE RUN. Then it writes a summary and apologises, while seven items that needed nothing from you sit untouched.

Underneath, it collapses one blocked item into a blocked run, and treats writing a report as a finishing move. THE APOLOGY IS THE TELL, because an apology fills the space where the next action should be.

Three things help. Put the skip rule in CLAUDE.md rather than only in the message, so it survives every session. Ask for the next action rather than a status update. And check real output, not what it says it did.

Instructions placed in project memory persist across sessions, where an instruction given in one message does not.

Claude Code docs, Memory and CLAUDE.md, checked September 2026

An agent that stops safely is not the same as an agent that does the work, so autonomy is a property to test for rather than assume.

First principles, not vendor documentation

Before you let it run unattended

1. Is the work committed, given the undo does not cover bash edits?
2. Does anything in this task write files through a shell command?
3. Are there symlinks in the tree I would need to restore by hand?
4. Is the skip rule in CLAUDE.md, or only in this message?
5. Did I ask for the next action, or for a status update?
6. Am I checking real output, or reading a report of it?
7. Should this search be a subagent so the evidence stays out of my context?
8. Is settings.local.json in .gitignore?
9. Which of my team standards is still only in my head rather than in a skill?
10. If it stops early, will I be able to tell that it did?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

☕ Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

Checkpointing docs

<https://code.claude.com/docs/en/checkpointing>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-14 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.